

ARC 500

Programming with Python and Generative AI

Scripting, Data Analysis, Visualization, Problem-Solving, and Optimization

Syllabus | Fall 2026

School of Architecture, Syracuse University

Contact Information

Instructor	Junho Chun, PhD
Office Location	Slocum Hall 306A
Email	jchun04@syr.edu
Phone	315.443.2808
Office Hours	Monday and Wednesday, 10:30 am–12:00 pm, and by appointment. If my office door is open, please feel free to drop by anytime.

Course Information

Lecture / Studio	Monday and Wednesday, 12:45 pm to 2:05 pm
Location	Slocum Hall 307
Prerequisite	None. This course is designed for beginners with no prior coding experience.
Credit	3 credit hours
Required	A laptop, brought to <i>every</i> class meeting (see <i>Required Laptop and Software</i> below)

Course Objectives

This course introduces Python programming from the ground up, integrating generative AI as a practical tool for learning and developing code. Students begin with core programming concepts—syntax, data types, logic, and functions—and progressively apply these skills to filter and screen data, create visualizations, solve optimization problems, and explore introductory machine learning. The emphasis is on building genuine programming literacy: students learn to write, read, debug, and extend code, with AI serving as an accelerator rather than a replacement.

Beyond writing scripts, the course provides a foundation in the theoretical principles that underpin computational problem-solving. Students explore the conceptual and mathematical foundations of key topics, including optimization theory (objective functions and constraints), machine learning fundamentals (classification, regression, and model evaluation), and data analysis methodology. This dual emphasis ensures that students not only know *how* to implement solutions in code but also understand *why* those methods work and *when* to apply them. Through a hands-on approach combining instruction and practice, the course prioritizes professional application over pure theoretical computer science, ensuring students gain experience that is immediately applicable across disciplines.

As generative AI reshapes the landscape of programming, the primary objective of this course shifts away from the memorization of syntax. Rather, the curriculum is designed to cultivate computational thinking, problem formulation, and the evaluation of AI-generated outcomes. Students

who integrate core programming fundamentals with AI literacy will be equipped with the resilience and adaptability necessary to navigate and leverage future technological advancements.

Learning Goals

After taking this course, students will be able to:

- Write, run, read, trace, test, and debug Python scripts that use core values and types, expressions, conditionals, loops, lists, dictionaries, and functions; classify errors as syntax, runtime, or logic errors and explain every retained line of code.
- Build reproducible NumPy, pandas, and GeoPandas workflows that document provenance and units; inspect data quality; clean, group, aggregate, and export data; and justify missing-data and outlier decisions.
- Select an appropriate visual encoding and produce labeled, accessible, and interpretable static, animated, and spatial evidence using Matplotlib and GeoPandas.
- Formulate design decisions as bounded optimization problems; implement linear, nonlinear gradient-based, and heuristic methods; compare methods; analyze sensitivity; and independently verify feasibility and constraints.
- Fit and evaluate supervised prediction models against baselines, diagnose limitations and leakage, integrate an evaluated model with optimization in a predict–optimize–decide workflow, and disclose and verify all generative-AI assistance used in the process.

Course Description

An introductory, application-focused course in Python and responsible generative-AI-assisted coding. Students build reproducible workflows for data analysis, visualization, spatial evidence, optimization, and evaluated machine learning, culminating in a defensible predict–optimize–decide design recommendation.

Course Delivery Format

The course is delivered in an in-person format, and the lectures will not be recorded. Each week has two meetings with distinct roles:

- **Meeting A (Monday)** introduces the week’s concepts and reasoning, opening with a 3–5 minute debrief of the most common error from the previous week’s submission.
- **Meeting B (Wednesday)** is a hands-on Spyder studio. Students build that week’s deliverable in class, with the instructor in the room. **This is why a laptop is required at every meeting.**

Three weeks depart from that two-meeting pattern, because of the academic calendar:

- **Week 3** — Monday, September 7 is Labor Day. Meeting A is replaced by a short asynchronous primer (approximately 40 minutes) to be completed before that Wednesday.
- **Week 8** — Monday, October 12 is Fall Break. There is no Meeting A and no substitute is assigned; that week’s only work is finishing Project 1.
- **Week 15** — Monday, December 7 is the only meeting. December 9 is a reading day, so the capstone clinic, synthesis, and in-class presentations all occur in that single session.

The University's Thanksgiving Break runs Sunday, November 22 through Sunday, November 29 and removes both ARC 500 meetings that week. It is treated as an unnumbered break between Weeks 13 and 14, so the numbered sequence stays 1–15.

Key Fall 2026 Dates

August 24	First day of classes and first ARC 500 meeting
August 31	Add deadline
September 14	Academic/financial drop, grading-option, and religious-observance notification deadline
October 12–13	Fall Break; ARC 500 does not meet Monday, October 12
November 20	University withdrawal deadline
November 22–29	Thanksgiving Break; no class November 23 or 25
December 7	Final ARC 500 meeting; Project 2 presentations and due date
December 9	Reading day; no class

Required Laptop and Software

Every student must bring a laptop to every class meeting. Both meetings each week involve live, hands-on work: reading and tracing code, running and repairing scripts, inspecting variables and output, and building the week's studio deliverable. A student without a working laptop cannot participate in the in-class activity that constitutes most of the graded work in this course.

- **Spyder IDE and Python 3.** Follow the current *Spyder Installation and Course Environment Guide* posted on Blackboard. Week 1 verifies the Editor, Console, Variable Explorer, working directory, and course environment. Use the posted guide's version-specific instructions rather than an installation command found elsewhere.
- **Course libraries.** The course uses NumPy, pandas, Matplotlib, GeoPandas, SciPy, and scikit-learn. Libraries are introduced and verified when they are first needed; exact installation and troubleshooting instructions are provided in the setup guide and weekly materials.
- **Generative AI coding assistant.** The course will identify at least one no-cost option. Students are not required to purchase a paid AI service. Use of any assistant is governed by the Artificial Intelligence Policy below.

If your laptop cannot run Spyder, or if you do not have regular access to a laptop, contact the instructor in the first week so an arrangement can be made before the graded studio work begins.

Course Expectations

The following guidelines are designed to facilitate a comfortable and productive learning environment.

Student Expectations	Instructor Expectations
Be punctual for all classes	Listen to and respect students' views
Bring a laptop to every class meeting, charged and able to run Spyder	Be available for questions at least 5 minutes before and after class
Read and review class notes prior to the lesson	Do not go beyond the allotted class time
Be active and engaged in class discussions and studio work	Accommodate differences in students' learning methodologies
Use laptops and devices for course work during class, not for unrelated activity	Create an active learning environment to facilitate student learning
Listen to and respect others	Reply to e-mails within 36 hours on weekdays and 48 hours on weekends
Complete all weekly studio assignments and both projects	Assign work that reflects the material covered in class
Submit all work before the stated deadline, with a complete AI-use record	Return feedback in time to inform the next week's work

If unable to meet the above expectations, contact the instructor ahead of time.

Note the deliberate difference from a lecture-only course: laptops are *required* here, not discouraged. The corresponding expectation is that the laptop is used for the course's own work during class time.

Attendance

Each student has an educational obligation to attend all classes. Attendance will be checked for every meeting. Because Wednesday studios build the graded deliverable in class with instructor support present, an absence from a Meeting B is materially harder to recover from than a missed lecture.

Arrival more than 10 minutes after the start of class is recorded as late; two late arrivals count as one absence. Arrival more than 15 minutes late or departure more than 15 minutes before the end of class counts as an absence. The first two absences carry no numeric attendance penalty. This two-absence buffer is separate from the determination that an absence is excused. Students who miss class remain responsible for notes, announcements, and assigned work. Four or more unexcused absences will trigger a required conference about the student's ability to complete the course successfully and, when timely, the option to withdraw before the University deadline.

Per University policy, Barnes Center at the Arch staff do not provide medical excuse notes for students. If Barnes Center staff determine it is medically necessary to remove a student from classes, they will coordinate with the case management staff directly to provide absence notification to faculty through Orange SUccess. For absences lasting less than 48 hours, students are encouraged to discuss academic arrangements directly with their faculty. Additional information may be found at Student Outreach and Retention: Absence Notifications.

University Expectation Regarding Attendance

Attendance in classes is expected in all courses at Syracuse University. Students are expected to arrive on campus in time to attend the first meeting of all classes for which they are registered. Students who do not attend classes starting with the first scheduled meeting may be academically withdrawn as not making progress toward degree by failure to attend. Instructors set course-specific policies for absences from scheduled class meetings in their syllabi. Students should also review the university's religious observance policy and make the required arrangements at the beginning of each semester.

Determination of Excused / Non-excused Absence

Whether an absence is excused is determined by the instructor after reviewing the available University notification or supporting information. Students should communicate promptly when an emergency, religious observance, University-authorized activity, disability-related need, or other serious circumstance affects attendance. An Orange SUccess absence notification communicates that an absence occurred; it does not by itself determine whether the absence is excused or waive missed course requirements. Approved accommodations and the University's Religious Observances Policy remain in effect.

Penalty for Unexcused Absences

After the two-absence buffer, each additional unexcused absence results in a 1.5-point deduction from the final course score. For example, three unexcused absences produce one 1.5-point deduction; the first two do not produce a numeric penalty.

Weekly Studio Assignments

A weekly assignment in this course *is* that week's Wednesday studio handout, completed and submitted as the graded artifact—not a second, separately designed deliverable layered on top. Work begins in the Wednesday studio with instructor support in the room; the polished, independently completed version is submitted afterward.

Every studio handout uses the same four-heading # %% cell structure, and that structure is the rubric skeleton:

QUESTION / INPUTS/ASSUMPTIONS / METHOD / CHECKS-INTERPRET

Submission Rhythm and Deadlines

- **Standard rule:** due before the *following* Monday's Meeting A, at **9:00 am**. That Monday then opens with a 3–5 minute debrief of the most common error before new content begins. Weeks 1, 3, 4, 5, 6, 9, 10, 11, and 12 follow this rule unchanged.
- **Three exceptions**, where the following Monday has no meeting to submit before and no session in which to debrief. Each is due the **Saturday before, 11:59 pm**, and each debrief moves to the next session that actually meets:
 - **Week 2** (studio Wed Sep 2) → due Saturday, September 5. The following Monday is Labor Day.
 - **Week 7** (studio Wed Oct 7) → due Saturday, October 10. The following Monday is Fall Break.
 - **Week 14** (studio Wed Dec 2) → due Saturday, December 5. The following Monday is Project 2's own due date and presentation day, so this gap preserves real runway between the last

checkpoint and the final submission.

- **One shifted deadline:** Week 13's assignment (studio Wed Nov 18) is due Monday, November 30 at **12:45 pm**, the start of Meeting A, because Thanksgiving break intervenes.
- **Weeks 8 and 15** carry no separate weekly assignment—their studio time *is* the project deliverable. This is why the weekly bucket is built from 13 graded weeks (1–7 and 9–14), not 15.

Late Policy

15% off within 24 hours late; zero credit after 48 hours, barring a prior arrangement made in advance of the deadline. Technical difficulties such as a poor internet connection are not considered an emergency justifying an extension. Students are required to keep an electronic record of each submission and to confirm that Blackboard returned a submission receipt. Incomplete submission will be considered non-submission.

Lowest Two Dropped

The two lowest weekly studio grades are dropped, so one bad week is not catastrophic.

The Transfer Safeguard

Starting in Week 4, every submission's **CHECKS-INTERPRET** section must include at least one self-generated check applied to a slightly different input than the one worked through live in class—a different row, case, parameter value, or small scenario tweak—with 2–4 **assert** statements the student writes and runs before submitting. Copying the in-class values alone does not satisfy this requirement.

Individual Work

All weekly assignments are individual. Discussing main concepts or general approaches is acceptable; joint problem-solving, sharing files, or submitting work substantially derived from another student's is not. Both students implicated—the one who provided the work and the one who replicated it—share equal responsibility for a breach of academic integrity under University policy. Using or disseminating solutions from prior years' assignments is likewise a violation.

Projects

This course has **no midterm or final exam**. In their place are two cumulative projects, each with milestones that *are* the ordinary weekly studio task for that week rather than additional artifacts layered on top. From Week 5 onward, students apply each week's studio skill directly to their own project data.

Project 1 — Evidence Before Design (assigned September 14; due October 14)

A building-performance and site data story built from the student's own chosen dataset. Deliverables: one or two **.py** files (**analysis.py** and, if needed, **evidence.py**); one cleaned CSV and one grouped-summary CSV; two static PNG figures plus either one map PNG or one animation GIF; a 600–800-word illustrated evidence memo; a README, data dictionary, and AI-use record; and a six-slide pin-up, using the provided template, presented in the Week 8 clinic.

The six-slide pin-up takes place during class on Wednesday, October 14. The complete digital submission is due in Blackboard by **11:59 pm Eastern Time on Wednesday, October 14, 2026**. Week 7's map remains a graded weekly studio. For the final Project 1 evidence extension,

students may submit either that map or the Week 6 animation; students choosing the animation still complete the Week 7 spatial studio as weekly course work.

Project 1 rubric	Weight
Question, provenance, cleaning, checks, and cleaned export	20%
Grouped evidence and defensible outlier decision	20%
Two purposeful static figures	20%
Chosen extension: map OR animation	15%
Interpretation, limitations, and bounded recommendation	15%
Reproducibility, code clarity, README, and AI-use record	10%
Total	100%

Milestones (each is that week’s regular studio assignment): Week 5 — dataset, research question, and provenance, via the grouping and outlier analysis; Week 6 — visualization checkpoint; Week 7 — spatial studio plus the compiled draft of Parts I–III already produced in Weeks 4–6.

Project 2 — Predict, Optimize, Decide (assigned October 19; due December 7)

Performance-informed design alternatives, integrating a fitted prediction model into an optimization on the student’s own problem. Deliverables: a one-page formulation and sensitivity memo; two or three .py files (`model_and_evaluate.py`, `optimize_and_integrate.py`, and an optional helper); one model metric table, one diagnostic figure, and one model card; one primary optimization result, one compact solver comparison, one hand-verified constraint table, and one solver card; four purposeful figures (sensitivity/design space, model diagnostic, solver comparison, and baseline versus recommended design); a 1,000–1,200-word report including the integration step and its limitations; a README and AI-use record; and a six-minute presentation plus a two-minute code/model walk-through, delivered in the Week 15 session.

The presentation and code/model walk-through take place during class on Monday, December 7. The complete digital submission is due in Blackboard by **11:59 pm Eastern Time on Monday, December 7, 2026**.

Project 2 rubric	Weight
Formulation, family justification, and sensitivity evidence	15%
Evaluated prediction, diagnostic, and model card	20%
Primary optimization and constraint verification	20%
Compact solver / baseline comparison	15%
Predict–optimize–decide integration	20%
Communication, reproducibility, limitations, and AI-use record	10%
Total	100%

Milestones (each is that week’s regular studio assignment): Week 10 — topic, dataset, and pairing proposal via the `linprog` formulation of the student’s own linear sub-problem; Week 11 — nonlinear formulation; Week 12 — heuristic-search track; Week 13 — baseline model, fitted, evaluated, and leakage-audited; Week 14 — evaluated prediction and model card.

Grades

The course evaluation is built from the weekly studio assignments and the two projects. There is no separate quiz or exam category, and no separately weighted participation line.

Component	Weight
Weekly studio assignments (13 graded weeks; lowest 2 dropped)	40%
Project 1 — Evidence Before Design (due Week 8)	25%
Project 2 — Predict, Optimize, Decide (due Week 15)	35%
Total	100%

Two policies are deliberately *not* weighted line items, and instead act on the composite score:

- **Attendance and professionalism** applies as a flat deduction on the final composite score, as described under *Penalty for Unexcused Absences*.
- **AI-use disclosure** is graded inside each weekly assignment's and each project's own rubric, and additionally functions as a compliance gate: a missing, incomplete, or fabricated disclosure caps that submission's score at **70%** regardless of technical quality.

A zero-prior-experience cohort meeting twice weekly for 80 minutes benefits most from a short, frequent, recoverable feedback loop rather than a few high-stakes tests. Hence 40% is earned in small weekly steps with the two lowest dropped, and the remaining 60% rewards cumulative integration. If students are expected to use AI coding assistants throughout, a memorized-syntax exam measures the wrong thing; the weekly assignments provide the frequent, individually attributable check instead.

Grade Scale

A grade “A” merits work that pushes to levels of excellence beyond the expectations of the class. A grade “B” merits very good work that fulfills the expectations of the class. A grade “C” merits work that fulfills the minimum requirement for the class. Work that does not fulfill the minimum expectations of this graduate-level course will receive a grade “F.” Syracuse University does not permit D or D– grades in graduate courses; ARC 500 uses the graduate grading standard for all enrolled students.

Numerical scale (all numbers round to the nearest whole; e.g. 68.52 = 69 = C while 68.33 = 68 = C–):

Percentage	Letter	Percentage	Letter
93 and above	A	77–80	B–
89–92	A–	73–76	C+
85–88	B+	69–72	C
81–84	B	65–68	C–
Below 65	F		

Course Materials

There is no required textbook to purchase for this course. Class notes, the weekly studio handouts, and the posted setup guide are the primary materials, and every graded artifact is fully specified in those materials on its own.

A **companion text** written for this course is available as a free, living document: *Introduction to Programming with Python* (J. Chun), an architecture-framed Python text that uses the same room, facade, beam, and grid examples as the lecture material. Nothing in this course depends on a student reading it; it is offered as optional deeper reading where a week's lecture cites a chapter. One deliberate divergence: the companion text installs and edits in VS Code, while this course uses Spyder throughout—follow the course's Spyder instructions where the two differ.

As of August 2026, the optional weekly references are: Week 1, Chapters 1 and 8; Week 2, Chapter 2; Week 3, Chapters 3–6; Week 4, Chapters 10–12; Week 5, Chapter 12; Week 6, Chapter 13; Week 7, Chapters 15–16; Week 9, Chapter 11; Week 11, Chapter 18; Week 12, Chapter 20; Week 13, Chapter 19; and Week 14, Chapter 21. Weeks 8, 10, and 15 have no matching assigned chapter. Chapter 22 (Neural Networks) is beyond the scope of ARC 500. Because this is a living document, the week-specific link posted on Blackboard is the reference of record if chapter numbering changes.

Reference Documentation

Official documentation is the reference of record for every library used, and learning to read it is itself a course skill:

- Python: <https://docs.python.org/3/>
- Spyder: <https://docs.spyder-ide.org/current/>
- NumPy: <https://numpy.org/doc/stable/>
- pandas: <https://pandas.pydata.org/docs/>
- Matplotlib: <https://matplotlib.org/stable/>
- GeoPandas: <https://geopandas.org/en/stable/>
- SciPy optimize: <https://docs.scipy.org/doc/scipy/reference/optimize.html>
- scikit-learn: <https://scikit-learn.org/stable/>

Course Role in the Program

ARC 500 is a 500-level Selected Topics course offered for graduate-level credit. This section is a three-credit elective. Its application to a student's B.Arch., M.Arch., or M.S. degree requirements depends on the student's program and any required faculty or advisor approval. Students should confirm degree applicability with their academic advisor.

Course Contents

1. Computational thinking and the Python execution model

- Decomposition, abstraction, and automation
- Problem formulation before syntax
- Scripts, statements, comments; syntax, runtime, and logic error
- The generative-AI-as-collaborator protocol
- The Spyder IDE: Editor, Console, Variable Explorer, kernel, tracebacks

2. Values, types, and precise expression

- `int`, `float`, `str`, `bool`, `None`
- Naming with units; operator precedence; unit conversion
- f-strings and formatted output; floating-point display
- Comparisons; `assert` as a first correctness check

3. **Decisions, repetition, and reusable logic**
 - `if` / `elif` / `else`; boundary cases; compound conditions
 - `for` loops over a list; the accumulator pattern
 - Lists and a minimal dictionary lookup
 - `def`, parameters, defaults, `return`; guard clauses
4. **The toolchain and data as evidence**
 - NumPy arrays: shape, dtype, indexing, vectorized arithmetic
 - pandas Series and DataFrame; the trustworthiness interview
 - Data dictionaries, provenance, and NaN as a modeling decision
 - Docstrings and type hints; filtering, derived columns, and CSV export
5. **Grouping, outliers, and defensible summaries**
 - Split-apply-combine; `groupby()`, multiple aggregations, sorting
 - IQR versus z -score outlier rules and where they disagree
 - Measurement error versus genuine extreme versus a missing variable
 - One key-based merge
6. **Visual reasoning: static and dynamic evidence**
 - Matching a question to an encoding: distribution, comparison, relationship, change over a sequence
 - The `fig`, `ax` pattern; labeled axes with units; legends
 - Annotating the one point that carries the argument
 - Dynamic evidence: `FuncAnimation` saved as a shareable GIF
7. **Spatial data, scale, and maps**
 - Geometry versus attributes; point, line, polygon; `GeoDataFrame`
 - CRS and EPSG codes; why area and length require a projected CRS
 - Buffer and spatial join; a properly labeled static choropleth
8. **Parameter sweeps and sensitivity**
 - Vectorized operations versus explicit loops
 - `arange` / `linspace`; boolean masks; `meshgrid`
 - Reading a swept surface: which input moves the outcome most
9. **Optimization I: formulation and exact linear methods**
 - Design variables, objective, constraints, bounds, feasible region
 - Standard form; why a linear problem is solved exactly rather than searched
 - `scipy.optimize.linprog`; hand-verified constraint tables
10. **Optimization II: nonlinear and gradient-based search**
 - `scipy.optimize.minimize`; initial guess and bounds
 - Local versus global optima; multi-start comparison
11. **Optimization III: heuristic and metaheuristic search**
 - Discrete and combinatorial choices; non-smooth objectives
 - Random search, hill-climbing with restarts, genetic algorithms (selection, crossover, mutation), simulated annealing
 - Comparison against a brute-force baseline, in result and runtime

12. Machine learning I: evaluated prediction (regression)

- Features versus target; supervised regression; `train_test_split`
- Why a naive baseline comes first; MAE, RMSE, R^2 ; residual plots
- Overfitting, underfitting, and data leakage—found and fixed in AI-drafted code

13. Machine learning II: classification, model cards, and decisions

- Classification as a decision rule; the confusion matrix
- Precision versus recall; choosing a threshold from real consequences
- `cross_val_score`; the model card as a deliverable

14. Synthesis: predict, optimize, decide

- A fitted model as a surrogate objective inside an optimization
- Reproducibility and the AI-use record
- Stating a bounded recommendation with its limitations

Blackboard

This class will use Blackboard for all materials relevant to the course, including handouts, studio files, assignment specifications, solutions, links to external materials, and grades. Blackboard should be visited frequently to obtain the latest versions of class notes, assignments, and reference materials. All deadline times for submitting materials on Blackboard are Eastern Time.

For Blackboard or account support, contact Information Technology Services at help@syr.edu, 315.443.2677, or the ITS Service Center in 1-227 CST, Life Sciences Complex.

Blackboard Discussions.

Use Blackboard Discussions for questions likely to help the class. Do not post a complete working solution to a current assignment; posting the exact error message and the shortest relevant code excerpt, together with what you already tried, is encouraged.

Academic Drop Deadline

The academic/financial deadline to drop a class is Monday, September 14, 2026. This is also the grading-option deadline and the religious-observance notification deadline. The University withdrawal deadline is Friday, November 20, 2026. A course withdrawal after the drop deadline and by the withdrawal deadline remains on the transcript with the grading symbol WD.

Syracuse University Policies

The following required statements reflect Syracuse University's current syllabus guidance.

Disability Statement

Syracuse University values diversity and inclusion and is committed to a climate of mutual respect and full participation. There may be aspects of the instruction or design of this course that result in barriers to your inclusion and full participation. I invite students to meet with me to discuss strategies and/or accommodations (academic adjustments) that may be essential to your success in collaboration with the Center for Disability Resources (CDR).

To discuss disability-accommodations or register with CDR, visit disabilityresources.syr.edu, or contact the center at 315.443.4498 or disabilityresources@syr.edu for more detailed information.

CDR is responsible for coordinating disability-related academic accommodations and will work with you to develop an access plan. Since accommodations may require early planning and generally are not provided retroactively, please contact CDR as soon as possible.

If any aspect of the required in-class laptop work presents an access barrier, please contact me in the first week so an arrangement can be made before the graded studio work begins.

Academic Integrity Policy

As a preeminent and inclusive student-focused research institution, Syracuse University places academic integrity at the forefront of learning and considers it a core value and guiding pillar of education. The University's Academic Integrity Policy provides guidelines for completing academic work with integrity.

Students are required to meet both course-specific and Universitywide academic integrity expectations, such as crediting your sources, doing your own work, communicating honestly and supporting academic integrity.

Upholding academic integrity includes the protection of faculty intellectual property. Students should not upload, distribute or share an instructor's course materials, including presentations, assignments, exams or other evaluative materials, without permission.

Using websites that charge fees or require uploading of course material (e.g., Chegg, Course Hero) to obtain exam solutions or assignments completed by others and presenting them as your own violates academic integrity expectations in this course and may be classified as a Level 3 violation.

All academic integrity expectations that apply to in-person assignments, quizzes and exams also apply online. Students found in violation of the policy are subject to grade sanctions determined by the course instructor and non-grade sanctions determined by the school or college offering the course. Students may not drop or withdraw from courses in which they face a suspected violation. Any established violation in this course may result in course failure regardless of violation level.

Artificial Intelligence Policy

Full Use with Disclosure and Citation. Based on the assignments and specified learning outcomes, the full use of artificial intelligence as a tool, with disclosure and citation, is permitted in this course.

Generative AI is permitted in this course, and using it well is one of the course's learning goals. This is a deliberate, course-specific policy: ARC 500 treats an AI coding assistant as a collaborator whose output must be verified, and much of the course's value lies in learning to formulate problems precisely, evaluate what an assistant returns, and repair what it gets wrong. Several assignments explicitly require auditing AI-drafted code and fixing a real defect in it.

Permission is conditional on all of the following. Each is a requirement, not a suggestion:

1. **You must be able to explain every line you submit.** You are responsible for being able to trace, test, modify, and explain any submitted line on request. "The assistant wrote it" is not an explanation, and inability to explain your own submission is treated as an academic integrity concern regardless of whether the code runs.
2. **You must keep and submit an AI-use record.** Every weekly assignment and both projects require a disclosure stating which tool was used, what was asked of it, what it returned, what you kept, what you changed or rejected and why, and how you tested each retained suggestion. A missing, incomplete, or fabricated disclosure **caps that submission at 70%** regardless of technical quality. A fabricated disclosure—claiming you did not use a tool that you did use, or inventing a testing process you did not perform—is a dishonesty violation, not merely a

documentation lapse.

3. **You must verify, not trust.** AI-generated code in this course is treated as a draft to be checked. Verification means running it, testing it against known values with `assert`, and confirming that the numbers, units, and interpretation are correct. Submitting unverified output that happens to run is not acceptable work.
4. **AI does not substitute for the reasoning being assessed.** Where an assignment asks you to interpret a result, defend a decision, state a limitation, or write a memo, that reasoning must be your own. Using a tool to draft prose you then adopt without independent judgment defeats the purpose of that component and will be graded accordingly.
5. **AI use does not license collaboration.** Assignments remain individual work. Sharing prompts, transcripts, or generated solutions with other students is the same violation as sharing hand-written code.
6. **Do not submit confidential or third-party material to an AI tool.** Do not upload course materials that are the instructor's intellectual property, other students' work, or any data you are not permitted to share.

If you are unsure whether a particular tool or feature is considered generative AI, or whether a particular use is acceptable, ask the instructor *before* using it. Asking is always acceptable and is never penalized.

Faith Tradition Observances

Syracuse University's Religious Observances Policy recognizes the diversity of faiths represented in the campus community and protects the rights of students, faculty and staff to observe religious holy days according to their traditions. Under the policy, students are given an opportunity to make up any examination, study, or work requirements that may be missed due to a religious observance, provided they notify their instructors no later than the academic drop deadline.

For any observances occurring before the academic drop deadline, students must notify faculty at least two academic days in advance. Students register their observances using MySlice.

Discrimination or Harassment

Syracuse University does not discriminate and prohibits harassment or discrimination related to any protected category including creed, ethnicity, citizenship status, reproductive health decisions, national origin, sex, gender, pregnancy, disability, marital status, age, race, color, veteran status, military status, religion, sexual orientation, domestic violence status, genetic information, gender identity and/or gender expression, shared common ancestry or any other status protected by applicable law.

Any complaint of discrimination or harassment related to any of these protected bases should be reported to Sheila Johnson-Willis, associate vice president and chief equal opportunity and Title IX officer in the Office of Equal Opportunity, Inclusion and Resolution Services (621 Skytop Road, Suite 1001, Syracuse, NY 13244; equalopp@syr.edu; 315.443.4018). She is responsible for coordinating compliance efforts under various laws including Titles VI, VII, IX and Section 504 of the Rehabilitation Act.

Use of Class Materials

Original class materials (lectures, notes, syllabus, study guides, handouts, assignments, studio files, projects, programs, and tools) of this course are the intellectual property of the course instructor (Junho Chun) and are protected by U.S. copyright law and by University policy. The course instructor is the exclusive owner of the copyright in those materials. Students may take notes and make copies of course materials for their own use. However, students should not provide these materials to other parties (i.e., websites, social media, or any student not registered or enrolled in this course) without written permission from the instructor. Doing so is a violation of intellectual property law and of the Code of Student Conduct.

This includes uploading course materials into a generative AI tool that retains or trains on submitted content.

Student Work

I intend to use academic work that you complete this semester in subsequent semesters for educational purposes. Before using your work for that purpose, I will either get your written permission or render the work anonymous by removing identifying material. The School of Architecture may also request digital versions of selected course work for its Student Works Archive. Selection criteria, file-naming protocols, and required formats will be explained during the semester; any required archive package will be evaluated within the applicable project's professional-submission and reproducibility criteria.

University Email Policy

Syracuse University email is the official channel for University communication. Students are responsible for monitoring their syr.edu accounts. Faculty and staff must use their official University email addresses when communicating with students about course matters; sending student records to a non-syr.edu address may create a FERPA violation. See the Syracuse University Email Policy.

Use of Turnitin

This class will use the plagiarism detection and prevention system Turnitin for its written deliverables (the Project 1 evidence memo and the Project 2 report). Students will have the option to submit their writing to Turnitin to check that all sources have been properly acknowledged and cited before final submission. The instructor will also submit these documents to Turnitin, which compares submitted documents against documents on the Internet and against student papers submitted to Turnitin at Syracuse University and at other colleges and universities. Students' knowledge of the subject matter of this course and their writing level and style will be considered in interpreting the originality of a report. Keep in mind that all written work you submit for this class will become part of the Turnitin.com reference database solely for the purpose of detecting plagiarism of such papers.

Course Schedule

Wk	Monday (Meeting A)	Wednesday (Meeting B)	Assignment & Note
1	Aug 24 Computational thinking and the Python execution model	Aug 26 Spyder installation and first program	Week 1 studio: trace and repair a starter script. Due Mon Aug 31, 9:00 am.
2	Aug 31 Values, types, and precise expression	Sep 2 Architectural quantity studio	Week 2 studio: unit-aware calculator with assert checks. Due Sat Sep 5, 11:59 pm (Labor Day exception).
3	Sep 7 <i>No class: Labor Day.</i> Asynchronous primer (~40 min) on decisions and functions, completed before Wednesday	Sep 9 Design-rule and function studio	Week 3 studio: validated function, loop, dictionary lookup, assert tests. Due Mon Sep 14, 9:00 am.
4	Sep 14 Meet the toolchain, then data as evidence	Sep 16 Trustworthiness studio	Project 1 assigned. Week 4 studio: cleaned dataset, data dictionary, drop-justification memo, plus one NumPy-derived column plotted as a single Matplotlib scatter. Due Mon Sep 21, 9:00 am.
5	Sep 21 Grouping, outliers, and defensible summaries	Sep 23 Grouping and outlier studio	Week 5 studio (= P1 milestone : dataset, question, provenance): grouped table, dual-rule outlier flags, decision sentences. Due Mon Sep 28, 9:00 am.
6	Sep 28 Visual reasoning: static and dynamic evidence	Sep 30 Visual evidence studio	Week 6 studio (= P1 visualization milestone): three static figures and one captioned animated GIF. Due Mon Oct 5, 9:00 am.
7	Oct 5 Spatial data, scale, and maps	Oct 7 Spatial join and choropleth studio	Week 7 studio (= P1 spatial layer + compiled draft): GeoPandas script, choropleth, CRS justification. Due Sat Oct 10, 11:59 pm (Fall Break exception).
8	Oct 12 <i>No class: Fall Break.</i>	Oct 14 Project 1 studio and evidence clinic	Project 1 pin-up in class; digital package due 11:59 pm. No separate weekly assignment; the clinic, pin-up, and submission are this week's work.
9	Oct 19 Parameter sweeps and sensitivity	Oct 21 Sweep and heatmap studio	Project 2 assigned. Week 9 studio: two-parameter sweep, heatmap, sensitivity finding, vectorized-vs-loop assert . Due Mon Oct 26, 9:00 am.
10	Oct 26 Optimization I: formulation and exact linear methods	Oct 28 linprog studio	Week 10 studio and P2 proposal : formulate your own linear subproblem with linprog and submit a hand-verified constraint table. Due Mon Nov 2, 9:00 am.

continued on next page

Course schedule, continued

Wk	Monday (Meeting A)	Wednesday (Meeting B)	Assignment & Note
11	Nov 2 Optimization II: nonlinear and gradient-based search	Nov 4 Multi-start optimization studio	Week 11 studio (= P2 nonlinear checkpoint): minimize run plus a multi-start comparison write-up. Due Mon Nov 9, 9:00 am.
12	Nov 9 Optimization III: heuristic and metaheuristic search	Nov 11 Genetic algorithm studio	Week 12 studio (= P2 heuristic checkpoint): GA or SA implementation versus a brute-force baseline. Due Mon Nov 16, 9:00 am.
13	Nov 16 Machine learning I: evaluated prediction (regression)	Nov 18 Regression studio	Week 13 studio (= P2 baseline-model checkpoint): baseline-vs-model metric table, residual plot, and a fixed leakage bug in AI-drafted code. Due Mon Nov 30, 12:45 pm , at the start of Meeting A (Thanksgiving shift).
–	Nov 23 <i>No class: Thanksgiving Break</i>	Nov 25 <i>No class: Thanksgiving Break</i>	Unnumbered break week; no new content.
14	Nov 30 Machine learning II: classification, model cards, and decisions	Dec 2 Classification / regression studio (track-dependent)	Week 14 studio (= P2 model-card checkpoint): track-appropriate model update, evaluation artifact, one-page model card. Due Sat Dec 5, 11:59 pm . Last week new skills are introduced.
15	Dec 7 Capstone clinic, synthesis, and in-class presentations (<i>the only meeting this week</i>)	Dec 9 <i>No class: reading day</i>	Project 2 presentations in class; digital package due 11:59 pm . No separate weekly assignment.

* Schedule subject to change. Any change will be announced in class and posted on Blackboard.

Calendar notes: classes begin Monday, August 24, 2026. Labor Day (Mon Sep 7) removes Week 3's Meeting A. Fall Break (Mon Oct 12 and Tue Oct 13) removes Week 8's Meeting A; Tuesday is not a class day for this Monday/Wednesday course. Thanksgiving Break runs Sun Nov 22 through Sun Nov 29 and removes both meetings that week, which is treated as an unnumbered break between Weeks 13 and 14 so the numbered sequence stays 1–15. The last ARC 500 meeting is Monday, December 7; December 9 is a reading day.